

Cybersecurity

System Management Basics

Lesson 4.1



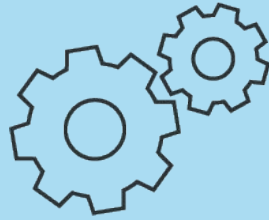
Learning Objectives

By the end of today's class, you will be able to:

- 1 Manage processes with the **top**, **ps**, and **kill** commands.
- 2 Install packages by using **apt install**.
- 3 Use newly installed programs such as terminator as a network professional.



Welcome



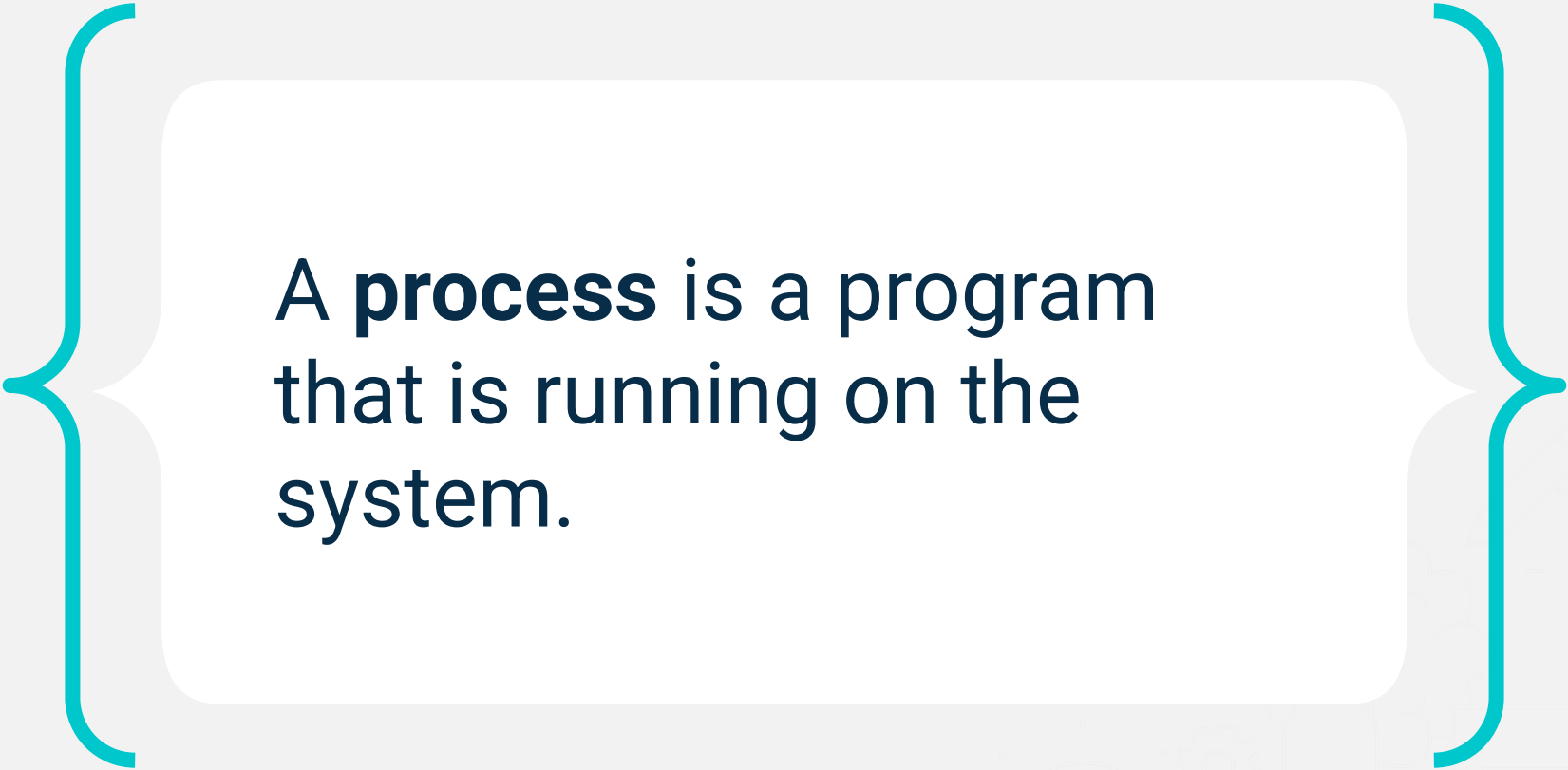
Resources and **Processes**

Resources and Processes

Scripts & Programs

- **Scripts** are executable programs that can run a system.
- Running programs on a system may:
 - Process data
 - Modify file system



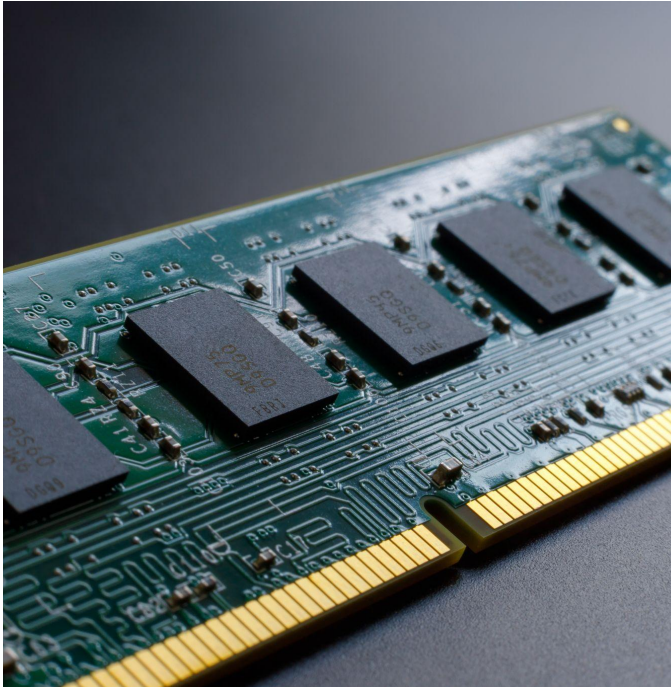


A **process** is a program
that is running on the
system.

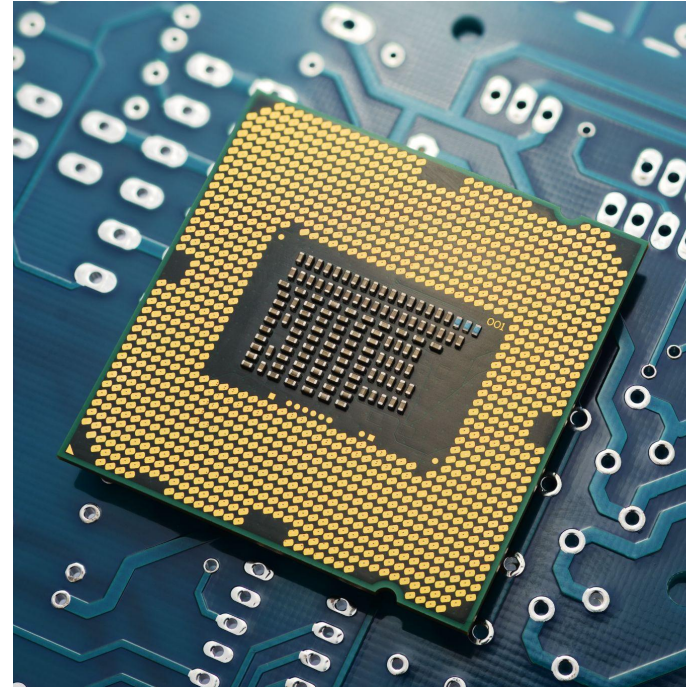


Programs that process, save, and modify data consume a computer's **resources**.

Memory



CPU



Memory

Memory comes in two forms:

Random Access Memory (RAM) is used to run the program's code.

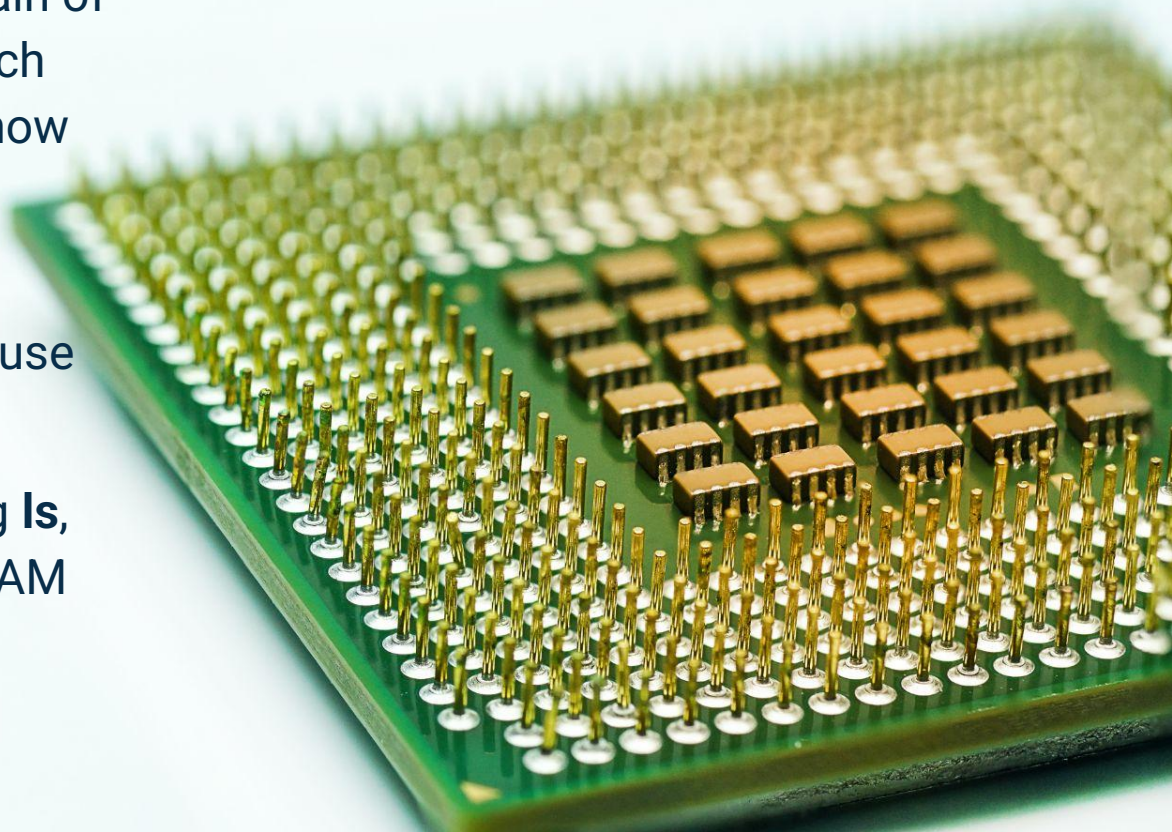
Disk Space is used to save data permanently.



Central Processing Unit (CPU)

The **CPU** acts as the central brain of a system, determining how much work a process has to do and how "difficult" that work is.

- A difficult task, such as encrypting a large file, will use a lot of **RAM** and/or **CPU**.
- Easier tasks, like executing **Is**, will require only minimal RAM and/or CPU.



Targeting Resources

Attackers can take advantage of a system's finite resources.

- Attackers can perform **denial of service (DoS)** attacks by launching processes that eat up memory on a target machine.
- This can slow down or crash the machine, making it unavailable to users, thus denying them service.



Targeting Resources

Attackers can also start malicious processes that don't use a lot of memory, and are therefore difficult to spot without specifically looking for them.

One example is a **backdoor process**.



This allows attackers to break into machines undetected.

These don't use much memory because they make a network connection to the attacker's machine, then listen for instructions.



Managing Processes

Linux has several commands for managing processes:



`top`



`ps`



`kill`



top

```
top - 19:39:51 up 0 min, 1 user, load average: 0.16, 0.04, 0.01
Tasks: 35 total, 1 running, 34 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.1 us, 0.1 sy, 0.0 ni, 99.8 id, 0.1 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 15679.2 total, 14827.1 free, 488.9 used, 363.1 buff/cache
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used. 14930.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	165728	11252	8364	S	0.7	0.1	0:00.55	systemd
454	root	20	0	43160	37100	9912	S	0.3	0.2	0:01.05	python3
2	root	20	0	2476	1436	1320	S	0.0	0.0	0:00.00	init-systemd(Ub
8	root	20	0	2504	144	132	S	0.0	0.0	0:00.00	init
43	root	19	-1	39472	14380	13396	S	0.0	0.1	0:00.09	systemd-journal
66	root	20	0	22100	6016	4608	S	0.0	0.0	0:00.11	systemd-udev
77	root	20	0	4492	204	56	S	0.0	0.0	0:00.00	snapfuse
78	root	20	0	4848	1780	1328	S	0.0	0.0	0:00.69	snapfuse
80	root	20	0	4492	156	12	S	0.0	0.0	0:00.00	snapfuse
86	root	20	0	4624	212	56	S	0.0	0.0	0:00.00	snapfuse
89	root	20	0	4492	192	48	S	0.0	0.0	0:00.00	snapfuse
92	root	20	0	4724	1740	1240	S	0.0	0.0	0:02.19	snapfuse
98	root	20	0	4492	168	24	S	0.0	0.0	0:00.00	snapfuse
99	root	20	0	4740	1684	1224	S	0.0	0.0	0:01.10	snapfuse
113	systemd+	20	0	25532	12396	8200	S	0.0	0.1	0:00.17	systemd-resolve
156	root	20	0	4304	2704	2464	S	0.0	0.0	0:00.00	cron
157	message+	20	0	8596	4672	4132	S	0.0	0.0	0:00.02	dbus-daemon
160	root	20	0	30104	19180	10396	S	0.0	0.1	0:00.13	networkd-dispat
161	syslog	20	0	222400	5332	4512	S	0.0	0.0	0:00.02	rsyslogd
162	root	20	0	1911648	36628	17680	S	0.0	0.2	0:00.38	snappd
177	root	20	0	15324	7176	6232	S	0.0	0.0	0:00.10	systemd-logind
227	root	20	0	4780	3280	3040	S	0.0	0.0	0:00.10	subiquity-serve
229	root	20	0	107228	21516	13392	S	0.0	0.1	0:00.09	unattended-upgr
235	root	20	0	3236	1072	984	S	0.0	0.0	0:00.00	agetty

- **top** allows you to observe all the running processes in real time.
- It is often used by administrators to get a general idea of how the system is running in real time.



ps

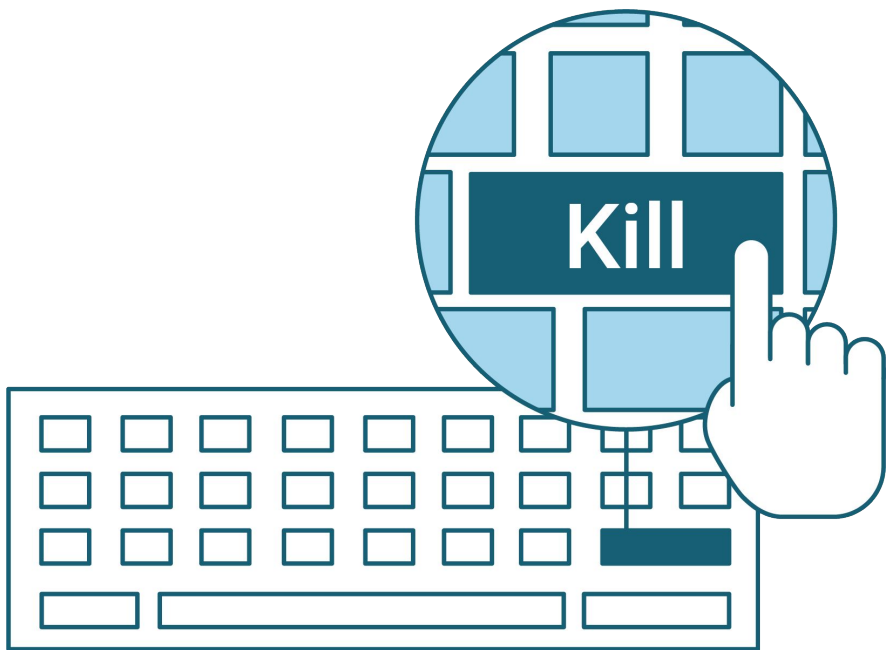
```
student:~$ ps
```

PID	TTY	TIME	CMD
6732	ttys002	00:00:00	/bin/bash
6867	ttys004	00:00:00	/usr/bin/ps
6897	pts/1	00:00:03	/bin/login -f

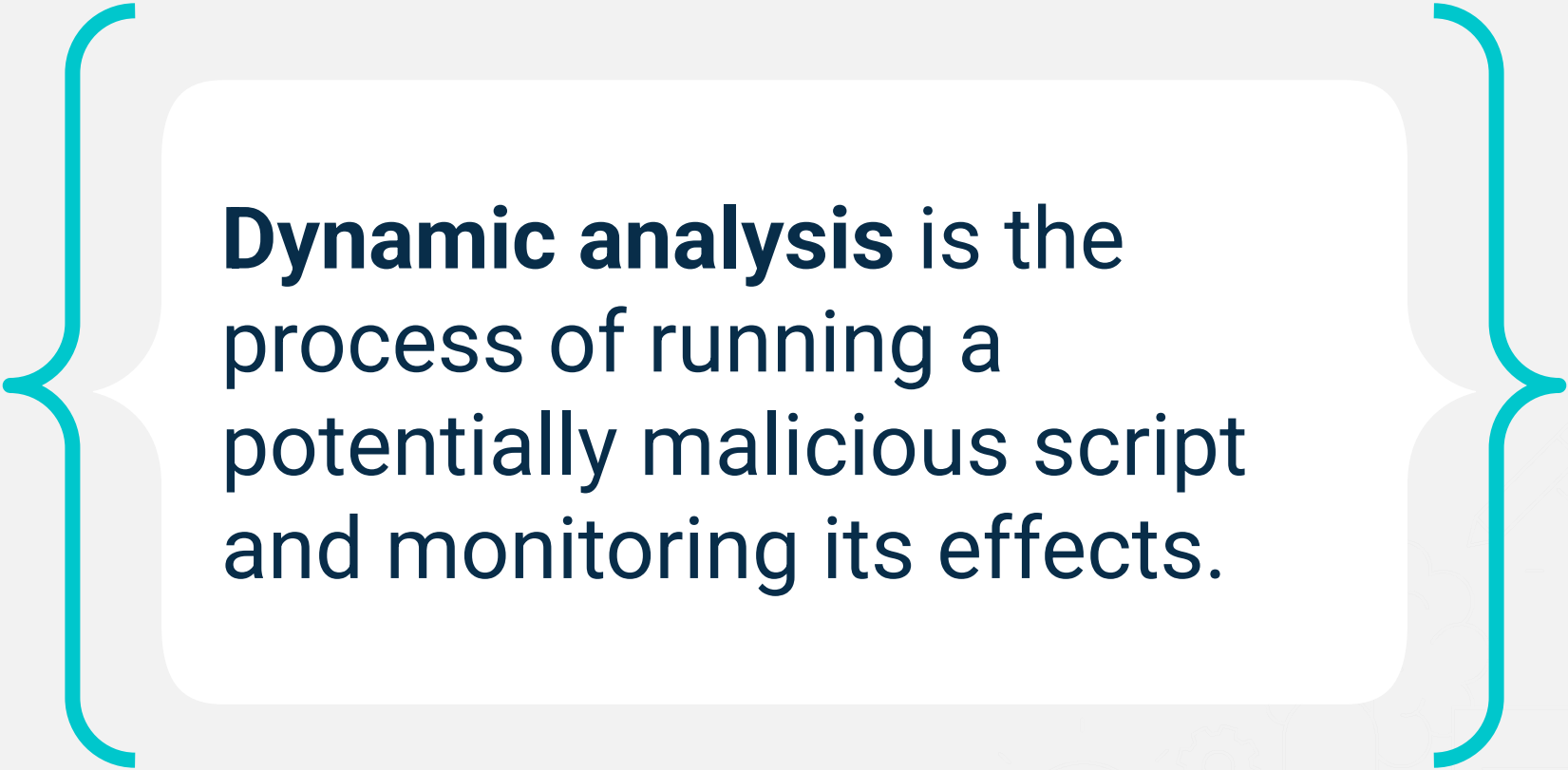
- `ps` allows you to take a snapshot of all the running processes on the system.
- It is often used with different arguments to observe different subsets of processes.



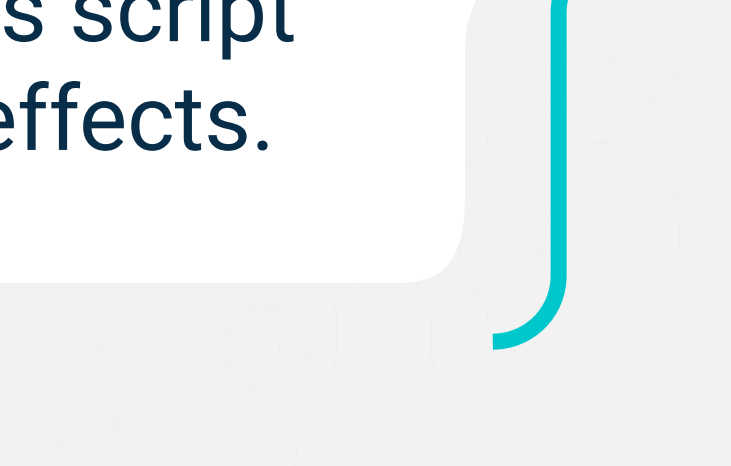
kill



- `kill` is used to **stop** processes.
- It attempts to allow a process to finish what it's doing before it shuts down.



Dynamic analysis is the process of running a potentially malicious script and monitoring its effects.

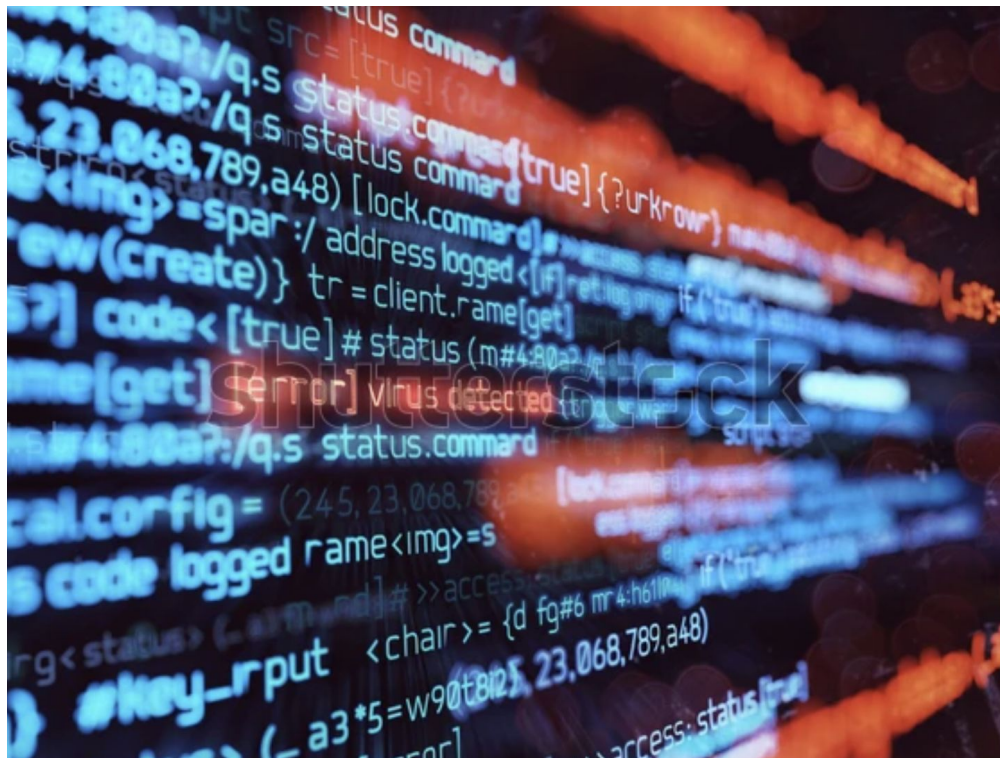


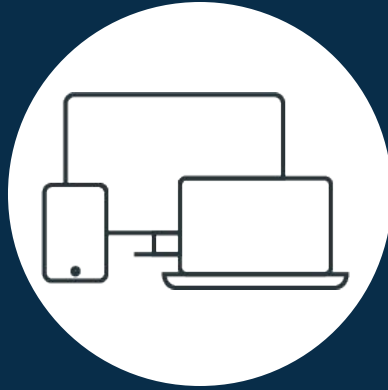


Demonstration Scenario

We will run the suspicious script we found in the **/tmp** directory during our previous audit.

- Use **top** and **ps** to understand what it does, and **kill** the process if needed.





Instructor **Demonstration**

Inspecting Malicious Files

Demo Summary

System Process Commands

top

Observes all the running processes. Displays information on:

- PID
- USER
- PR
- NI
- Command
- %CPU / %MEM



ps

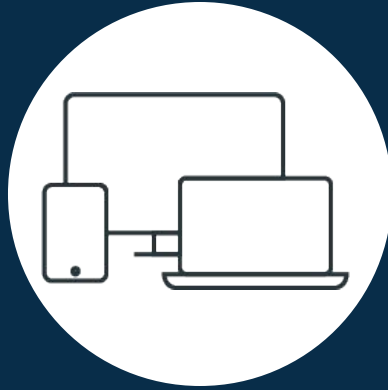
Takes a snapshot of all the running processes. Run with additional options to show additional information:

- -a
- -u
- -x





Now that we understand the information report **top**, we'll run our potentially malicious script to see if that information changes.



Instructor **Demonstration**

Executing Malicious Files

Demo Summary

Process Commands:

kill

Used to end processes. Syntax options include:

- `kill <PID>`
- `killall <process_name>`





Activity:

Process Investigation

Previously, you completed a basic audit of the system and found some malicious script files and a user that was not supposed to be on the system.

- You will investigate all the processes running on the system to determine if there are any obvious processes that should not be running.
- Your senior administrator has asked that you record snapshots of processes as well as review the processes in real time for anything suspicious.
- If you find any processes amiss, kill them and add your findings to your report.

Suggested Time:
25 Minutes



Time's up!
Let's review

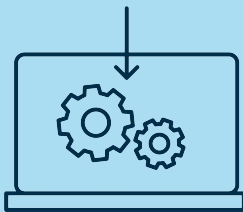


Questions?





Break
15 mins



Installing **Packages**



Packages

Administrators often install additional software to properly harden the machines they manage.

- Linux offers downloadable tools called **packages**.
- New packages are installed with a tool called **package manager**.
- We'll use the Ubuntu package manager, called **aptitude**, with the command `apt`.



Package Managers

Utilized through the terminal

01

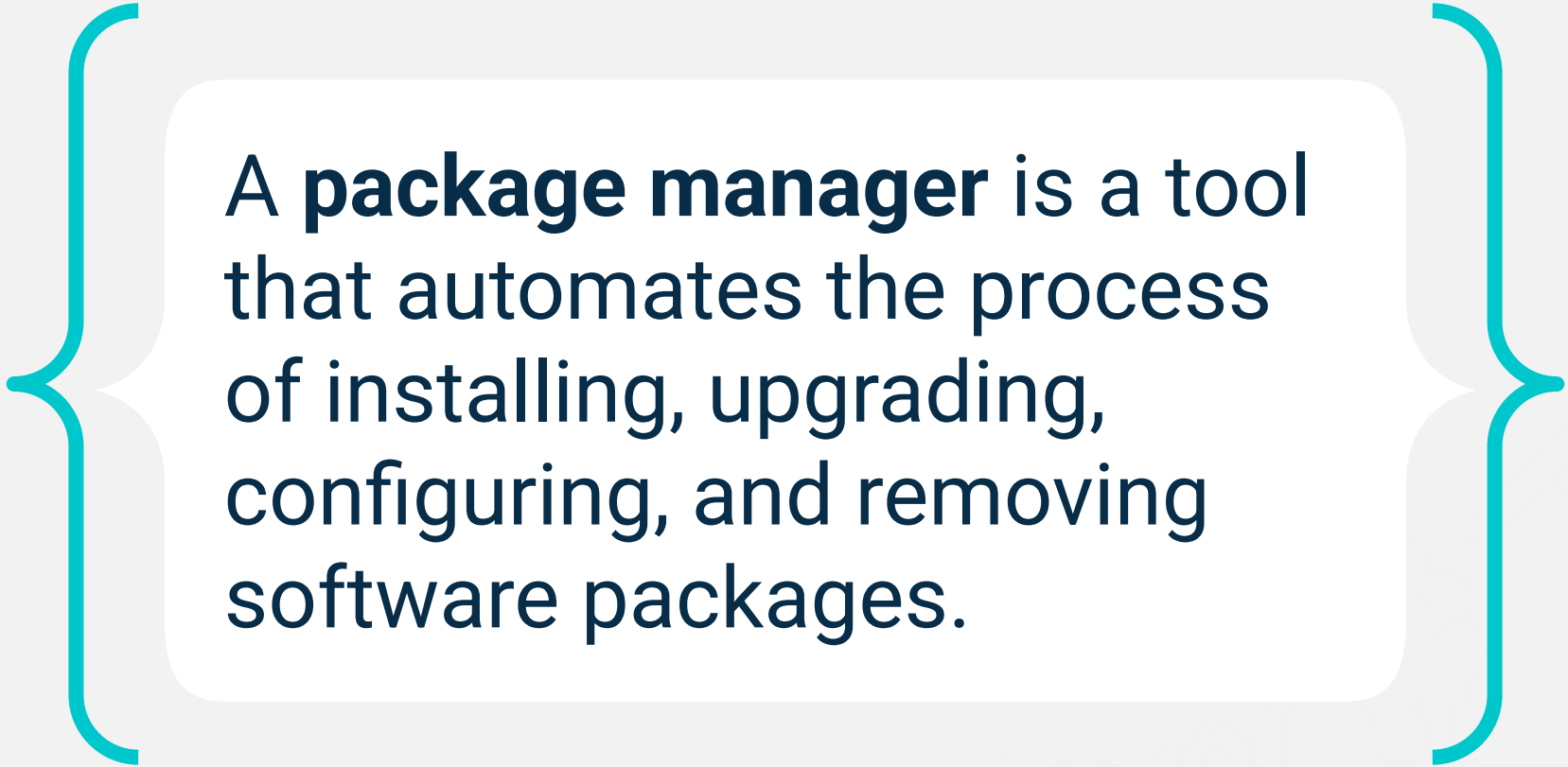
Simplify the installation and management of software

02

Handle software dependencies automatically

03

Centralized system for software updates and security patches



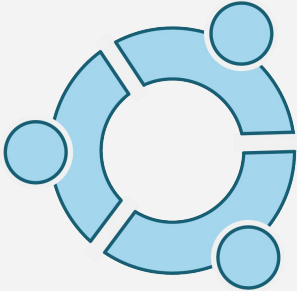
A **package manager** is a tool that automates the process of installing, upgrading, configuring, and removing software packages.

Package Managers

Various package managers help simplify software management

01

**Advanced Package
Tool (APT)**



02

**Yellowdog Updater
Modified (YUM)**



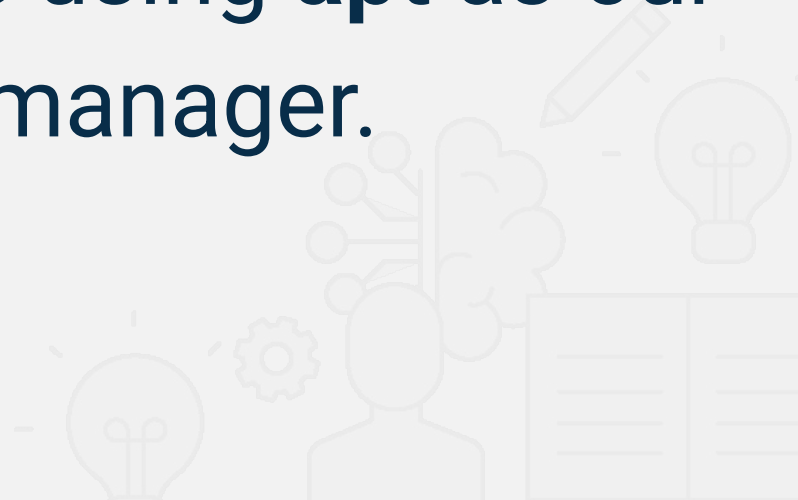
03

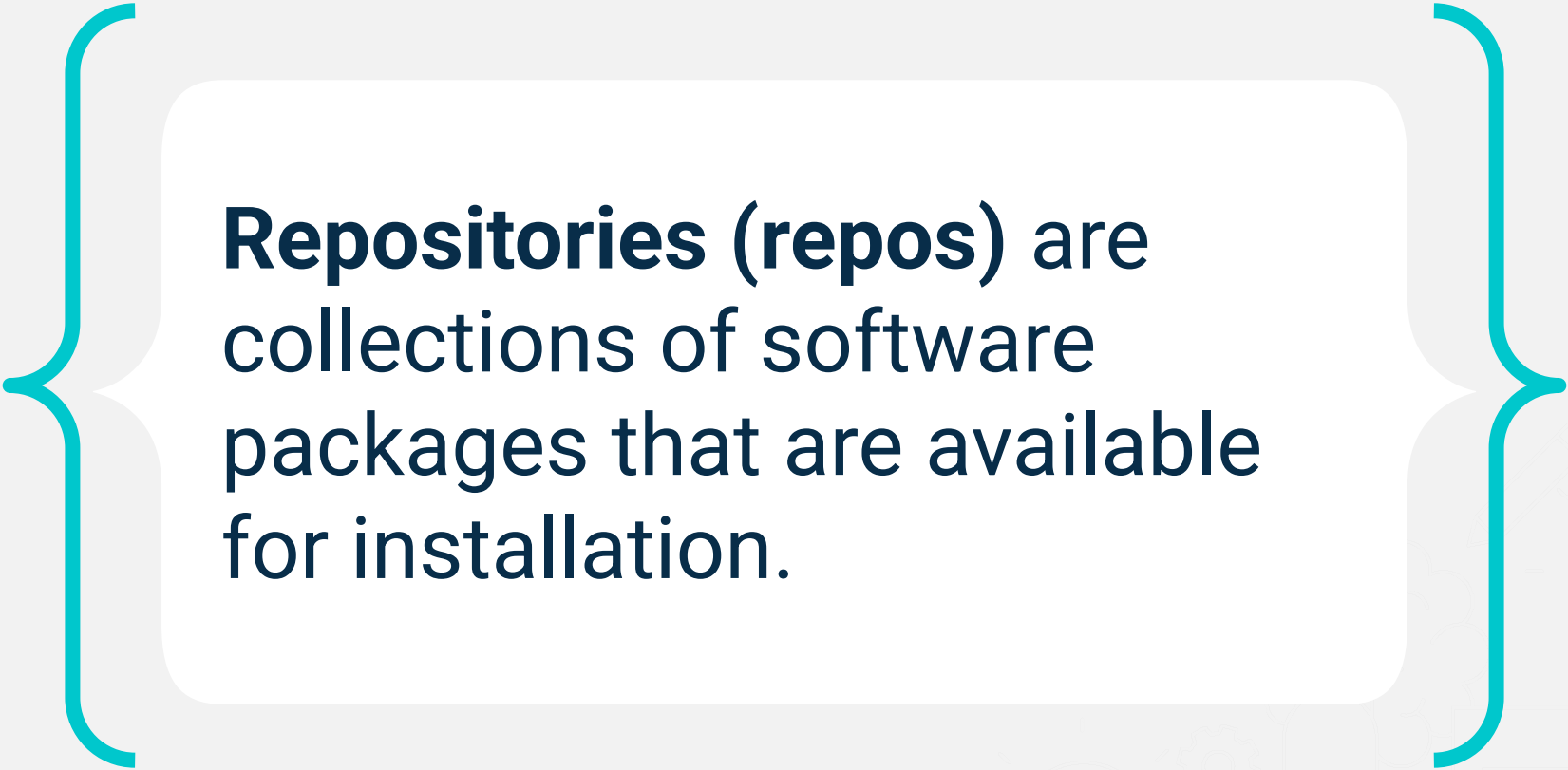
**PackageManager
(PACMAN)**





Since we are using Ubuntu,
we will be using **apt** as our
package manager.





Repositories (repos) are collections of software packages that are available for installation.

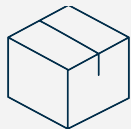


Repositories

Different types of repositories for installation

Official Repositories

Maintained by the Linux distribution.



Third-Party Repositories

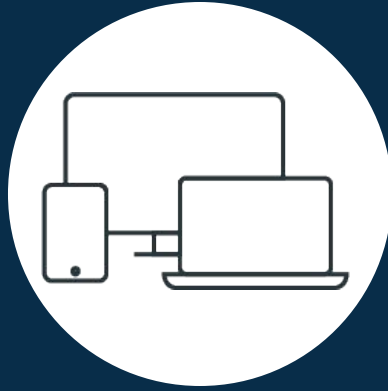
Maintained by other organizations or individuals.



Personal Package Archives (PPAs)

User-maintained repositories that can be added to the system.





Instructor **Demonstration**

Introduction to apt

Demo Review

Syntax options for `apt`

```
sudo apt update
```

Updates the list of available packages and their versions.

```
sudo apt upgrade
```

Installs the newest versions of all packages currently installed.

```
sudo apt install <package>
```

Installs a specified package.

```
sudo apt remove <package>
```

Removes a specified package.

```
sudo apt search <package>
```

Searches for a package in the repositories.

```
sudo apt install <package> -y
```

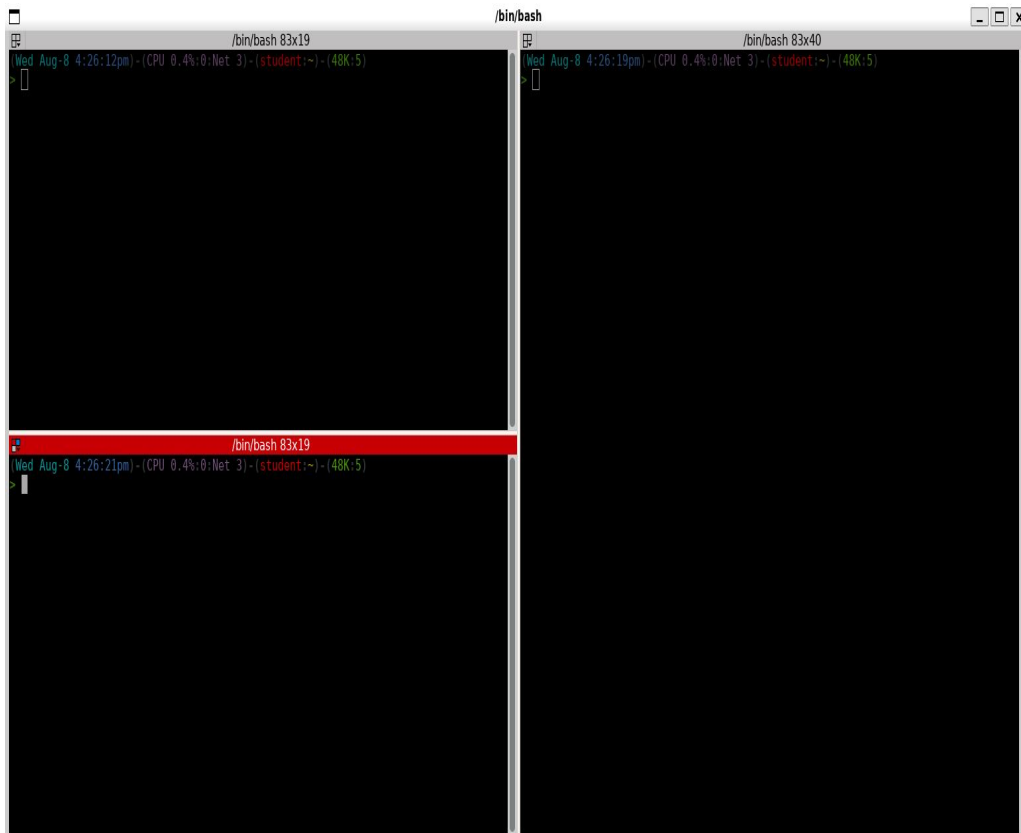
Automatically installs package without confirmation.



Demonstration Follow Along

While we have been doing everything in the default terminal window, this is not the only option for command line access in Linux.

- Terminal has limited functionality with **Single Window** and **Basic Features**.
- **Terminator** supports more advanced features:
 - Multiple planes
 - Tabs
 - Customization
 - Broadcasting
 - Search





Activity:

Installing Packages

Now that we have stopped a malicious service from running and completed a basic audit, we're going to cover some of the miscellaneous utilities that we can install onto our VMs.

- `emacs` is a traditional file editor.
- `cowsay` is a utility that takes in input and displays a cow repeating it.

Suggested Time:
20 Minutes



Time's up!
Let's review



Questions?





Recap

System Management Basics

01

Managed processes with the `top`, `ps`, and `kill` commands

02

Explored concepts of RAM and CPU components

03

Installed packages using `apt`

04

Worked with `terminator`, utilizing more advanced options



Next class

Audit passwords, elevate privileges, and user/group management



Thank you